

Escuela Politécnica Nacional

Facultad de Ingeniería de Sistemas

DOCUMENTACIÓN DE MANTENIMIENTO

CRUD pets-crud - Sistema de Gestión de Mascotas

Repositorio analizado: JonathanGuaman2004/YO-SOY-CRUD

Módulo: YO-SOY-CRUD/pets-crud

Tipo de informe: análisis y documentación de mantenimiento correctivo, preventivo, perfectivo y adaptativo

Fecha: mayo de 2026

Índice de contenidos

1. Introducción
2. Información general del sistema
3. Diagnóstico inicial del CRUD
4. Clasificación del mantenimiento aplicado
5. Cambios realizados en la versión corregida
6. Plan de pruebas
7. Evidencias recomendadas para el taller
8. Guía de ejecución paso a paso
9. Riesgos mitigados
10. Conclusiones y recomendaciones
11. Anexos

Índice de tablas

1. Tabla 1. Información técnica del módulo pets-crud.
2. Tabla 2. Diagnóstico inicial de problemas encontrados.
3. Tabla 3. Matriz de tipos de mantenimiento.
4. Tabla 4. Cambios realizados por archivo.
5. Tabla 5. Casos de prueba propuestos.
6. Tabla 6. Evidencias recomendadas.
7. Tabla 7. Comandos de ejecución y verificación.
8. Tabla 8. Matriz de trazabilidad problema-cambio-prueba.

1. Introducción

El presente documento describe el análisis y la documentación del mantenimiento aplicado al módulo pets-crud, perteneciente al repositorio YO-SOY-CRUD. El módulo corresponde a un sistema de gestión de mascotas que permite registrar, consultar, actualizar, eliminar y visualizar estadísticas básicas de mascotas. Además, se comunica con epn-event-manager para enviar eventos cuando se realiza una acción dentro del CRUD.

La documentación se elaboró con el mismo enfoque utilizado para epn-event-manager: primero se identifica el estado inicial del sistema, luego se clasifican los problemas encontrados según el tipo de mantenimiento, después se describen los cambios realizados y finalmente se plantea un conjunto de pruebas y evidencias que permitan demostrar que el mantenimiento fue aplicado correctamente.

El objetivo principal no es cambiar la lógica del sistema sin necesidad, sino mejorar su estabilidad, seguridad, documentación, facilidad de configuración y capacidad de prueba. Por ello, se consideran cuatro tipos de mantenimiento: correctivo, preventivo, perfectivo y adaptativo.

2. Información general del sistema

El módulo pets-crud está desarrollado con Node.js, Express, SQLite mediante node:sqlite, HTML, CSS y JavaScript. El servidor expone endpoints para el manejo de mascotas y sirve una interfaz web ubicada en public/index.html. La base de datos local se almacena en SQLite y se inicializa automáticamente si no existe.

Elemento	Descripción
Nombre del módulo	pets-crud
Propósito	Gestionar mascotas mediante operaciones CRUD y enviar eventos al epn-event-manager.
Tecnologías	Node.js, Express, SQLite, Axios, HTML, CSS, JavaScript, ESLint y node:test.
Puerto backend corregido	4002
Base de datos	SQLite local: db/pets.sqlite o ruta definida con PETS_DB_PATH.
Interfaz web	public/index.html
Servidor principal	server.js
Endpoints principales	GET /pets, GET /pets/:id, POST /pets, PUT /pets/:id, DELETE /pets/:id, GET /pets/stats, GET /health
Integración externa	epn-event-manager mediante EVENT_MANAGER_URL y EVENT_MANAGER_HEALTH_URL.

2.1 Funcionalidades principales

- Crear una mascota con nombre, especie, raza, edad, dueño, teléfono, estado y descripción.
- Listar todas las mascotas registradas en la base SQLite.
- Consultar una mascota por su identificador.
- Actualizar parcialmente o completamente los datos de una mascota.
- Eliminar una mascota existente.
- Consultar estadísticas de mascotas totales, activas, en tratamiento y perdidas.
- Enviar eventos de consulta, creación, actualización y eliminación al epn-event-manager.

3. Diagnóstico inicial del CRUD

Durante la revisión inicial se identificó que el sistema funcionaba como CRUD básico, pero presentaba inconsistencias y riesgos que justificaban aplicar mantenimiento. Los problemas encontrados no impedían por completo la ejecución del sistema, pero sí podían afectar su uso, documentación, seguridad, despliegue y evaluación mediante pruebas.

Problema detectado	Impacto	Tipo de mantenimiento sugerido	Prioridad
El README indicaba abrir	Puede confundir al usuario y	Correctivo	Alta

http://localhost:4001, mientras el servidor usa el puerto 4002.	hacerle pensar que el sistema no funciona.		
El frontend usaba una URL fija: http://localhost:4002.	Dificulta ejecutar el sistema en otro puerto, servidor o computadora.	Correctivo / Adaptativo	Alta
El script npm test existía, pero no había pruebas visibles para validar el CRUD.	No se podía demostrar de forma automática que las funciones principales siguen funcionando.	Preventivo / Correctivo	Alta
El servidor esperaba el envío de eventos al Event Manager antes de responder al usuario.	Si el Event Manager está apagado o lento, la operación del CRUD puede tardar más.	Correctivo / Preventivo	Media
CORS estaba completamente abierto con app.use(cors()).	En producción puede permitir solicitudes desde cualquier origen.	Preventivo	Media
El endpoint /health exponía la ruta física de la base de datos.	Puede revelar información interna innecesaria.	Preventivo	Media
Los datos ingresados por el usuario se mostraban mediante innerHTML.	Puede generar riesgo de inyección HTML o XSS si se ingresa contenido malicioso.	Correctivo / Preventivo	Alta
La documentación no detallaba variables de entorno ni conexión con epn-event-manager.	Dificulta instalar, probar o adaptar el sistema.	Perfectivo / Adaptativo	Media

4. Clasificación del mantenimiento aplicado

Con base en el diagnóstico, se concluye que al módulo pets-crud se le deben aplicar los cuatro tipos de mantenimiento. Esto permite justificar el trabajo realizado no solo como una corrección puntual, sino como una mejora integral del sistema.

Tipo de mantenimiento	¿Aplica?	Justificación	Ejemplo en pets-crud
Correctivo	Sí	Corrige errores, inconsistencias o comportamientos que pueden causar fallos.	Corrección del puerto documentado, API fija, riesgo de XSS y bloqueo por Event Manager.
Preventivo	Sí	Reduce la posibilidad de errores futuros y mejora la seguridad del sistema.	Pruebas automatizadas, CORS configurable, ocultamiento de ruta interna y validaciones.
Perfectivo	Sí	Mejora la calidad, claridad y experiencia sin cambiar el propósito principal del sistema.	README más completo, mejor guía de instalación, mejor manejo de estadísticas y mensajes.
Adaptativo	Sí	Permite que el sistema funcione en otros entornos o configuraciones.	Uso de variables de entorno, URL relativa en frontend y archivo .env.example.

5. Cambios realizados en la versión corregida

La versión corregida entregada modifica archivos existentes y agrega nuevos archivos para mejorar la mantenibilidad del CRUD. Estos cambios están orientados a conservar la funcionalidad original, pero reduciendo riesgos técnicos y facilitando la ejecución de pruebas.

Archivo	Cambio realizado	Tipo de mantenimiento	Beneficio
server.js	Se configuró x-powered-by, CORS configurable, timeout del Event Manager mediante variable, envío de eventos no bloqueante y respuesta /health sin exponer databasePath.	Correctivo / Preventivo / Adaptativo	Mejora seguridad, estabilidad y configuración.
public/index.html	Se reemplazó la API fija por window.location.origin, se agregó escapeHTML y se ajustó la carga de estadísticas desde backend.	Correctivo / Preventivo / Adaptativo	Permite usar el sistema en otros entornos y reduce riesgo XSS.
README.md	Se corrigió el puerto, se ampliaron instrucciones, endpoints, variables de entorno y mantenimiento aplicado.	Correctivo / Perfectivo	Evita confusión y mejora la documentación del proyecto.

.env.example	Se agregó plantilla de variables de entorno.	Adaptativo / Preventivo	Facilita configurar el sistema sin modificar código fuente.
test/server.test.js	Se agregaron pruebas con node:test para validaciones, normalización y ciclo CRUD básico.	Preventivo / Correctivo	Permite validar automáticamente que el CRUD funciona.
package.json	Se conserva la estructura de scripts: start, dev, check, lint y test.	Preventivo	Permite ejecutar verificaciones desde la terminal.

5.1 Corrección de puerto en documentación

El servidor utiliza el puerto 4002 por defecto, por lo que la documentación fue corregida para que el usuario abra la dirección correcta. Esta corrección evita que el sistema sea evaluado como fallido por una instrucción equivocada.

```
http://localhost:4002
```

5.2 Adaptación de la URL del frontend

Antes, el frontend dependía de una dirección fija. Esto funcionaba solo cuando el servidor estaba exactamente en localhost:4002. La versión corregida usa el origen actual del navegador, permitiendo que el sistema se adapte mejor a otros puertos o equipos.

```
const API = window.location.origin;
```

5.3 Protección del renderizado de datos

Se incorporó una función para escapar caracteres especiales antes de insertar datos de usuario en el HTML. Con esto se reduce el riesgo de que un texto ingresado por un usuario sea interpretado como código HTML o JavaScript dentro de la interfaz.

```
function escapeHTML(value) {
  return String(value ?? '')
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}
```

5.4 Envío de eventos sin bloquear la respuesta

El CRUD debe seguir funcionando aunque epn-event-manager esté apagado. Por eso, el envío de eventos se maneja de forma que no detenga la respuesta principal al usuario. Esto mejora la tolerancia a fallos y evita que operaciones simples como crear o listar mascotas se vuelvan lentas por un servicio externo.

5.5 Agregado de pruebas automatizadas

Se agregó un archivo de pruebas que usa node:test y assert. Las pruebas trabajan con una base SQLite temporal para evitar alterar la base real del proyecto. Esto permite validar el comportamiento del CRUD sin depender de datos existentes.

```
npm test
```

6. Plan de pruebas

Las pruebas propuestas combinan pruebas automatizadas y pruebas manuales. Las pruebas automatizadas permiten verificar reglas internas del backend, mientras que las pruebas manuales permiten demostrar que la interfaz funciona correctamente desde el navegador.

ID	Tipo	Caso de prueba	Resultado esperado
PU-01	Unitaria	Validar mascota sin nombre, especie inválida, dueño vacío, edad decimal y estado inválido.	El sistema devuelve errores de validación.
PU-02	Unitaria	Normalizar especie Perro y especie desconocida.	Perro se convierte a perro y especie desconocida se convierte a otro.
PU-03	Unitaria	Normalizar estado perdido y estado desconocido.	perdido se conserva y estado desconocido se convierte a activo.
PU-04	Unitaria / integración local	Crear mascota con datos válidos en SQLite temporal.	Se genera un ID con formato PET-0001 o similar.
PU-05	Unitaria / integración local	Buscar mascota por ID.	Se recupera la mascota creada.
PU-06	Unitaria / integración local	Actualizar nombre y estado de una mascota.	Los campos modificados se guardan correctamente.
PU-07	Unitaria / integración local	Consultar estadísticas después de crear y actualizar.	El total y los contadores por estado son coherentes.
PU-08	Unitaria / integración local	Eliminar mascota existente.	La mascota deja de existir en la base temporal.
PM-01	Manual	Abrir http://localhost:4002 .	Se muestra la interfaz del Sistema de Gestión de Mascotas.
PM-02	Manual	Crear una mascota desde el formulario.	La mascota aparece en el listado y se actualizan las estadísticas.
PM-03	Manual	Editar una mascota desde el listado.	El formulario carga los datos y permite guardar cambios.
PM-04	Manual	Eliminar una mascota.	La tarjeta desaparece del listado.
PM-05	Manual	Probar búsqueda y filtro por estado.	El listado se filtra correctamente.
PM-06	Manual	Consultar <code>/health</code> .	El backend responde status ok y estado del hub.

6.1 Casos de aceptación en formato Dado/Cuando/Entonces

CA-01 Crear mascota válida: Dado que el usuario se encuentra en la pantalla principal del CRUD, cuando completa nombre, especie y dueño y presiona Guardar mascota, entonces el sistema registra la mascota y la muestra en el listado.

CA-02 Validar campos obligatorios: Dado que el usuario intenta registrar una mascota, cuando deja vacío el nombre, la especie o el dueño, entonces el sistema muestra un mensaje indicando que esos campos son obligatorios.

CA-03 Editar mascota existente: Dado que existe una mascota registrada, cuando el usuario presiona Editar, modifica datos y guarda, entonces el sistema actualiza la información sin crear un registro duplicado.

CA-04 Eliminar mascota: Dado que existe una mascota registrada, cuando el usuario presiona Eliminar y confirma la acción, entonces el sistema elimina la mascota del listado.

CA-05 Filtrar mascotas: Dado que existen mascotas con diferentes estados, cuando el usuario selecciona un filtro de estado, entonces el sistema muestra solo las mascotas que coinciden con ese estado.

CA-06 Funcionamiento sin Event Manager: Dado que `epn-event-manager` no está disponible, cuando el usuario crea, actualiza o elimina una mascota, entonces el CRUD debe completar la operación y registrar el problema solo como evento no enviado.

7. Evidencias recomendadas para el taller

Para demostrar el mantenimiento realizado, se recomienda tomar capturas antes y después de aplicar la versión corregida. Estas evidencias deben mostrar tanto el funcionamiento del sistema como la ejecución de comandos de verificación.

N.º	Evidencia	Dónde tomar la captura	Qué debe verse
1	Estructura del proyecto corregido	Explorador de archivos o VS Code	Archivos <code>server.js</code> , <code>public/index.html</code> , <code>README.md</code> , <code>.env.example</code> y carpeta <code>test</code> .
2	Instalación de dependencias	Terminal dentro de <code>pets-crud</code>	Ejecución de <code>npm install</code> sin

			errores.
3	Verificación de sintaxis	Terminal	npm run check finalizado correctamente.
4	Lint	Terminal	npm run lint sin errores críticos.
5	Pruebas automatizadas	Terminal	npm test mostrando pruebas aprobadas.
6	Servidor iniciado	Terminal	Mensaje indicando que el sistema corre en http://localhost:4002.
7	Interfaz principal	Navegador	Pantalla Sistema de Gestión de Mascotas.
8	Crear mascota	Navegador	Mascota nueva visible en el listado.
9	Editar mascota	Navegador	Datos actualizados en la tarjeta.
10	Eliminar mascota	Navegador	La mascota ya no aparece en el listado.
11	Endpoint /health	Navegador o Postman	Respuesta JSON con status ok.
12	Event Manager	epn-event-manager	Evento recibido o estado offline sin afectar el CRUD.

8. Guía de ejecución paso a paso

Los siguientes pasos permiten aplicar y validar la versión corregida del CRUD.

1. Descargar el archivo pets-crud-corregido.zip.
2. Descomprimir el ZIP.
3. Copiar el contenido de la carpeta pets-crud sobre la carpeta original YO-SOY-CRUD/pets-crud.
4. Abrir una terminal en la carpeta YO-SOY-CRUD/pets-crud.
5. Ejecutar npm install.
6. Ejecutar npm run check.
7. Ejecutar npm run lint.
8. Ejecutar npm test.
9. Ejecutar npm start.
10. Abrir el navegador en http://localhost:4002 y probar el CRUD.

Comando	Propósito
cd YO-SOY-CRUD/pets-crud	Ingresar a la carpeta del módulo.
npm install	Instalar dependencias.
npm run check	Revisar sintaxis de server.js.
npm run lint	Ejecutar análisis de estilo y calidad con ESLint.
npm test	Ejecutar pruebas automatizadas con node:test.
npm start	Iniciar el servidor.
http://localhost:4002	Abrir la aplicación en el navegador.

9. Riesgos mitigados

La versión corregida reduce varios riesgos técnicos y de presentación académica. Estos riesgos son importantes porque pueden afectar tanto el funcionamiento real del sistema como la calificación del taller si no se cuenta con evidencias claras.

Riesgo antes del mantenimiento	Cambio aplicado	Riesgo después del mantenimiento
El usuario abre el puerto equivocado por una instrucción incorrecta.	README corregido a http://localhost:4002.	Bajo. La guía coincide con el servidor.
El CRUD solo funciona en localhost:4002.	Frontend adaptado con window.location.origin.	Bajo. Puede funcionar en otros entornos donde se sirva desde el mismo origen.
Datos maliciosos pueden renderizarse como HTML.	Escape de caracteres antes de pintar datos.	Medio-bajo. Se reduce el riesgo de XSS en listado.
El Event Manager apagado retrasa las operaciones CRUD.	Envío de eventos sin bloquear respuesta principal.	Bajo. El CRUD sigue funcionando.
No hay evidencia automática de que el CRUD funciona.	Pruebas con node:test.	Bajo. npm test permite comprobar funciones clave.

Variables de configuración no documentadas.	.env.example y README mejorado.	Bajo. La instalación es más clara.
---	---------------------------------	------------------------------------

9.1 Matriz de trazabilidad problema-cambio-prueba

Problema	Cambio realizado	Prueba o evidencia asociada
Puerto incorrecto en documentación	README corregido a puerto 4002	Captura del README y navegador en http://localhost:4002 .
API fija en frontend	Uso de <code>window.location.origin</code>	Prueba manual abriendo la interfaz desde el servidor.
Riesgo de XSS por innerHTML	Función <code>escapeHTML</code> aplicada a datos visibles	Crear mascota con caracteres <code><</code> <code>></code> y verificar que se muestran como texto.
Falta de pruebas	<code>test/server.test.js</code> con <code>node:test</code>	Captura de <code>npm test</code> aprobado.
Dependencia del Event Manager	Eventos no bloqueantes y <code>timeout</code> configurable	Probar CRUD con Event Manager apagado.
CORS abierto	<code>ALLOWED_ORIGINS</code> configurable	Revisar <code>server.js</code> y <code>.env.example</code> .

10. Conclusiones y recomendaciones

- El módulo `pets-crud` cumple con las operaciones principales de un CRUD de mascotas y puede integrarse con `epn-event-manager` mediante eventos.
- El análisis permitió identificar problemas reales de documentación, configuración, seguridad, pruebas y adaptabilidad.
- Sí se justifica aplicar mantenimiento correctivo, preventivo, perfectivo y adaptativo al módulo.
- La versión corregida mejora la calidad del sistema sin cambiar el objetivo funcional del CRUD.
- Las pruebas automatizadas agregadas permiten respaldar técnicamente el mantenimiento realizado.
- Para la entrega del taller se recomienda adjuntar capturas de comandos, interfaz, endpoints y funcionamiento del CRUD.

10.1 Recomendaciones futuras

- Separar el CSS y JavaScript del archivo `index.html` para mejorar la mantenibilidad.
- Agregar pruebas de integración HTTP usando herramientas como Supertest si el docente lo permite.
- Agregar paginación y filtros backend si el volumen de mascotas crece.
- Mantener el archivo `.env` fuera del repositorio y versionar solo `.env.example`.
- Configurar GitHub Actions para ejecutar `check`, `lint` y `test` automáticamente en cada push.
- Documentar con capturas cualquier cambio aplicado al código antes de subirlo al repositorio.

11. Anexos

11.1 Endpoints principales del CRUD

Método	Ruta	Descripción
GET	/	Carga la interfaz web del CRUD.
GET	/health	Verifica estado de API, base de datos y conexión con Event Manager.
GET	/pets/stats	Devuelve estadísticas básicas de mascotas.
GET	/pets	Lista todas las mascotas.
GET	/pets/:id	Consulta una mascota por ID.
POST	/pets	Crea una mascota.
PUT	/pets/:id	Actualiza una mascota existente.
DELETE	/pets/:id	Elimina una mascota existente.

11.2 Variables de entorno documentadas

Variable	Uso
PORT	Define el puerto del servidor. Valor recomendado: 4002.
PETS_DB_PATH	Permite cambiar la ruta de la base SQLite.
EVENT_MANAGER_URL	Define la URL donde se envían eventos del CRUD.
EVENT_MANAGER_HEALTH_URL	Define la URL para revisar el estado del Event Manager.
EVENT_TIMEOUT_MS	Define el tiempo máximo de espera para la integración externa.
ALLOWED_ORIGINS	Permite restringir los orígenes aceptados por CORS.

11.3 Texto sugerido para colocar en el informe académico

Se aplicó mantenimiento al módulo pets-crud con el objetivo de corregir inconsistencias, prevenir errores futuros, mejorar la documentación y permitir su adaptación a diferentes entornos de ejecución. El mantenimiento correctivo se enfocó en solucionar la inconsistencia del puerto, eliminar la dependencia de una URL fija en el frontend, reducir el riesgo de inyección HTML y evitar que la integración con epn-event-manager bloquee las operaciones principales del CRUD. El mantenimiento preventivo se evidenció mediante la incorporación de pruebas automatizadas, configuración de CORS y documentación de variables de entorno. El mantenimiento perfectivo se aplicó mediante la mejora del README y la organización de instrucciones de uso. Finalmente, el mantenimiento adaptativo se logró con la incorporación de .env.example y el uso de configuraciones externas para puerto, base de datos y URLs del Event Manager.